

Anonymous Inner class :

→ An inner class without a name called Anonymous class.

Q why its used?

→ When we want to override the behaviour of the method without even creating any subclass.

```
Ex: public abstract class Car {
    public abstract void pressBreak();
}
```

```
public class Test {
    sum() {
```

```
        Car audiCarObj = new Car() {
```

```
            @Override
```

```
            public void pressBreak() {
```

```
                // audi car specification impln here
```

```
                System.out.println("Audi specific break changes");
            }
        }
    }
}
```

```
    }
```

```
    } // end of stmt
```

```
    audiCarObj.pressBreak();
}
```

```
}
```

```
}
```

Note: What 2 things happened behind the scenes

- Sub class is created, name decided by the compiler.
- creates an object of subclass and assign its reference to object "audiCarObj"

final class it is.
compiler generate class name
ex Test\$1.class

Generic class

Let us understand why need of generic class? with an example.

→ As we know Object is parent of all subclass and when we can't know which type of variable will come or return, we explicitly use object as D.O.T due to which we extensively rely on typecasting to our desired D.O.T

Q: public class Point {

Object value;

public Object getPrimitiveValue() {

return value;

}

public void setPointValue(Object value) {

this.value = value;

}

public class Main {

psum() {

Point printObj = new Point();

printObj.setPointValue(1); // Passing Integer here

Object printValue = printObj.getPointValue();

// we can not use primitiveValue directly,

// we have to typecast it, else it will be

// compile time error.

if ((int) printValue == 1) {

// do something

}

}

}

Integer (Int)
String (String)
Class (ClassName)

Primitive
Reference
Object
as typecasting
due to object
reference

— Now the typecasting problem in the above scenario is resolved by Generic classes

→ Symbol used is diamond symbol (< >)

→ T can be defined dynamically at runtime
→ T can be any Non-primitive or

```
Ex: public class Point<T> {
    T value;
    public T getPointValue() {
        return value;
    }
    public void setPointValue(int T value) {
        this.value = value;
    }
}
```

Here we set coming variable is of type Integer, except Integer we pass any D.T throws error

```
public class Main {
    psum() {
        Point<Integer> pointObj = new Point<Integer>(0);
        pointObj.setPointValue(1);
        Integer printValue = pointObj.getPointValue();
        if (printValue == 1) {
            // do something
        }
    }
}
```

No typecasting
required here →

More than one generic types example

```

public class Pair<K, V> {
    private K key;
    private V value;

    public void put(K key, V value) {
        this.key = key;
        this.value = value;
    }
}

public class Main {
    psum() {
        new Pair<String, Integer>();
        Pair<String, Integer> pairObj = new Pair<>();
        pairObj.put("hello", 1234);
    }
}

```

both are right way to instantiate.

Generic Method:

What if we only want to make method generic, not the complete class, we can write Generic Methods too.

- Type parameter should be before the return type of the method declaration.
- Type parameter scope is limited to method only.

```

public class GenericType {

```

Ex:

define generic type before return type

```

    public <K, V> void printVal(Pair<K, V> pair1) {
        if (pair1.getKey().equals("Mohit")) {
            // do something
        }
    }
}

```


Raw Type: Its a name of the generic class or interface without any type arguments.

```
public class Print<T>{
```

T value;

```
public T getPrintValue(){  
    return value;  
}
```

```
public void setPrintValue(T value){  
    this.value = value;  
}
```

```
}
```

```
public class Main{  
    psum();
```

```
Print<String> paraTypePrintObj = new Print<>();
```

This is generic way

// when we use raw type compiler internally passes

// Object as parameter type.

```
Print rawTypePrintObj = new Print();
```

```
// Print<Object> rawTypePrintObj = new Print();
```

```
rawTypePrintObj.setValue(1);
```

```
rawTypePrintObj.setPrintValue("StringHello");
```

```
}
```

```
}
```

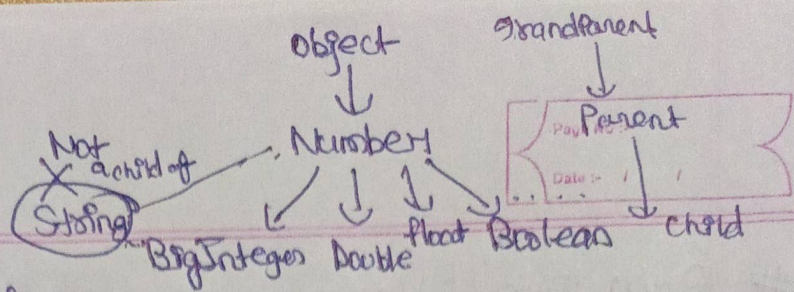
Integer

String

Same method accepting any type variable

Print

Compiler execute like this



Bounded Generics:

→ It can be used at generic class and method.

- Upper Bound ($\langle T \text{ extends } \text{Number} \rangle$) means T can be of type `Number` or its subclass only. Here `Superclass` (in this example `Number`) we can have interface too.
- Multi Bound.

Upper Bound:

ex:

```

public class Print<T extends Number> {
    T value;
    public T getPrintValue() {
        return value;
    }
    public void setPrintValue(T value) {
        this.value = value;
    }
}

```

can be `Number` or below it

```

public class Main {
    print() {

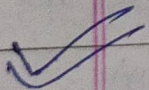
```

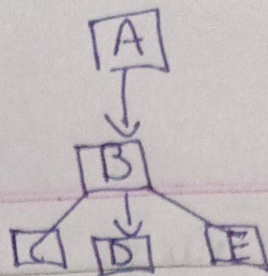
`<String>` → compilation error we get if passes `String`

```

        Print<Integer> paraTypePrintObj =
            new Print<Integer>();
    }
}

```





Wildcards:

- Upper bounded wildcard: i.e. class Name and below

Syntax: `<? extends UpperBoundClassName>`

- If UpperBoundClassName is B then it accepts: classes {B, C, D, E} ^{Not} ~~{A}~~

•

- lower bounded wildcard: `<? super LowerBoundClassName>`
i.e. class Name and above

- If LowerBoundClassName is B then it accepts: class {A, B} but not {C, D, E} X

- Unbounded wildcard `<?>` only you can read

↳ works on those method which are present on

Ex: public class Print {

Object → Size()

↳ generic method's present in object
then go with Unbounded wildcard
(get)

// wildcard method

```
public void computeList(List<? extends Number> src,
                        List<? extends Number> destn)
{.....}
```

// Generic type method

```
public <T extends Number> void computeList(
    List<T> src, List<T> destn) {.....}
```

9.


```
public class Main {
```

```
    psum() {
```

```
        List<Integer> wildcardIntList = new ArrayList<>();
```

```
        List<Float> wildcardFloatList = new ArrayList<>();
```

① diff

wildcard method

```
        Print printObj = new Print();
```

```
        printObj.computeList(wildcardIntList, wildcardFloatList);
```

can accept both

Integer & Float because child of Number

Generic method

cannot accept two types of

Variable because

<T> is either replaced by Integer or Float so in

that scenario where flexibility is required wildcard method

wins the race. Generic method used only parameter is 1 type

or all fixed typed var.

X printObj.computeList2(wildcardIntList, wildcardFloatList);
↳ throw compile time error.

```
}
```

```
}
```

② diff

② Generic <K, V, T> like that we have to use if we want to achieve this scenario but wildcard method we have only one "?" used.

Type Erasure: means when we compile generic class/methods or wildcard methods, Bytecode is generated in that Bytecode (.class file) all generics/wildcard got replaced with actual type.

Generic class Example

Ex 1.

replaced
with
value
in Bytecode
when generated

```
public class Point<T>{
```

```
    T value;
```

```
    public void setValue(T val){
```

```
        this.value = val;
```

```
    }
```

```
}
```

⇒

```
public class Point{
```

```
    Object value;
```

```
    public void setValue(Object v){
```

```
        this.value = v;
```

```
    }
```

Generic class Bound Type Example:

```
public class Point<T extends Number>{
```

```
    T value;
```

```
    public void setValue(T val){
```

```
        this.value = val;
```

```
    }
```

```
}
```

replaced
with
Number
here

⇒

```
public class Point{
```

```
    Number value;
```

```
    public void setValue(Number n){
```

```
        this.value = n;
```

```
    }
```

```
}
```